202044502
Programming with
JAVA

Unit -2
(Part -1)
Topic – Inheritance and Abstract

Academic Year: 2024-25 (II)

Prepared By:
Prof. Pratik B. Soni
(Asst. Prof. – I.T. Dept, MBIT)

Inheritance & Abstract

- Inheritance Basics
- "super" keyword
- Constructors in Inheritance
- Method Overriding
- Dynamic Method Dispatch
- Abstract Classes
- Object class

Inheritance & Abstract

- **Inheritance Basics**
- "super" keyword
- Constructors in Inheritance
- Method Overriding
- Dynamic Method Dispatch
- Abstract Classes
- Object class

Inheritance & Abstract – Inheritance Basics

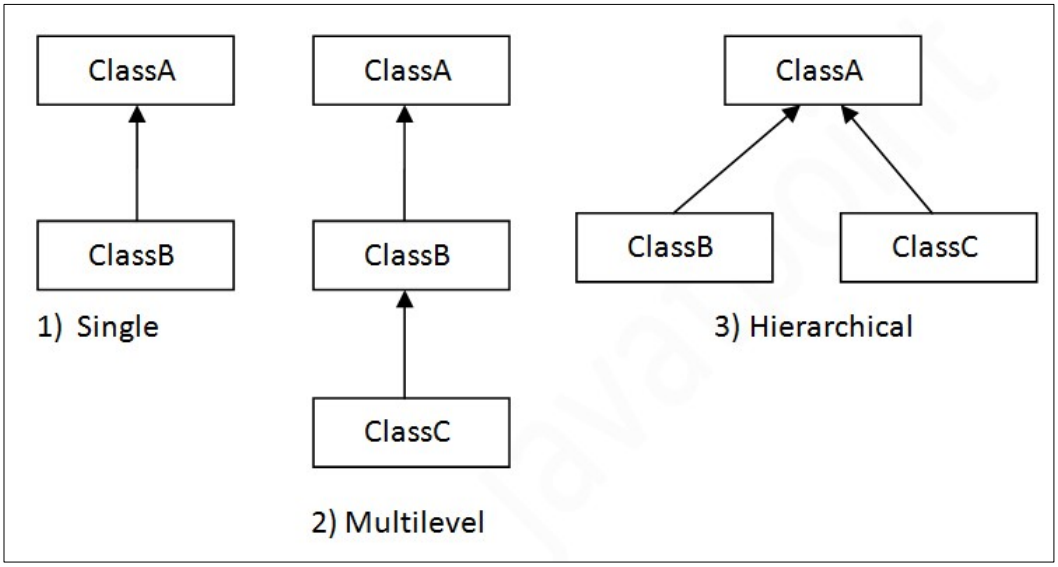
- Inheritance in Java is a mechanism in which one object acquires all the properties and behaviours of a parent object.
- Inheritance in Java enables a class to inherit properties and actions from another class, called a **superclass or parent class.**
- A class derived from a superclass is called a **subclass or child group.**
- Through inheritance, a subclass can access members of its superclass (fields and methods).

- **Syntax:**

```
class <Subclass-name> extends <Superclass-name>
{
        //methods and fields
}
```

Inheritance & Abstract – Inheritance Basics

- Inheritance Types - **Multiple inheritance is not supported in Java through class.**



5

Inheritance & Abstract – Inheritance Basics

- Inheritance examples

```
class A
{
        int i, j;
        void showij() {
        System.out.println("i and j: " + i + " " + j);
        }
}

class B extends A
{
        int k;
        void showk() {
        System.out.println("k: " + k);
        }
        void sum() {
        System.out.println("i+j+k: " + (i+j+k));
        }
}
```

```
class SimpleInheritance
{
public static void main(String args[]) {
        A superOb = new A();
        B subOb = new B();

        superOb.i = 10;
        superOb.j = 20;
        System.out.println("Contents of superOb: ");
        superOb.showij();

        subOb.i = 7;
        subOb.j = 8;
        subOb.k = 9;

        System.out.println("Contents of subOb: ");
        subOb.showij();
        subOb.showk();

        System.out.println("Sum of i, j and k in subOb:");
        subOb.sum();
        }
        }
```

OUTPUT

```
Contents of superOb:
i and j: 10 20
Contents of subOb:
i and j: 7 8
k: 9
Sum of i, j and k in subOb:
i+j+k: 24
```

6

Inheritance & Abstract – Inheritance Basics

- Inheritance examples
- Private members

```
class A
{
        int i;
        private int j;
        void setij(int x, int y)
        {
                i = x;
                j = y;
        }
}

class B extends A
{
        int total;
        void sum()
        {
                total = i + j;
        }
}
```

? error

Inheritance & Abstract –

```
class data
{
        int erno;
        String name = "";
        double cgpa;

        data()
        {
        System.out.println("from super"+"\n");
        }
        data(int e, String s, double c)
        {
                erno=e;
                name=s;
                cgpa=c;
        }
        void display()
        {
        System.out.println("erno:" + erno);
        System.out.println("name:" + name);
        System.out.println("cgpa:" + cgpa + "\n");
        }
}
```

```
class student extends data
{
        int mobile;
        student(int e, String s, double c, int m)
        {
                erno=e;     name=s;
                cgpa=c;     mobile=m;
        }
        void display()
        {
        System.out.println("sub_erno:" + erno);
        System.out.println("sub_name:" + name);
        System.out.println("sub_cgpa:" + cgpa);
        System.out.println("sub_mobile:" + mobile + "\n");
        }
}

class inh1
{
public static void main(String args[])
{
data d1 = new data();
student s1 = new student(01,"abc",8.5,99886);
s1.display();

d1 = s1;    // sub obj reference to super obj
d1.display();
}
}
```

```
from super

from super

sub_erno:1
sub_name:abc
sub_cgpa:8.5
sub_mobile:99886

sub_erno:1
sub_name:abc
sub_cgpa:8.5
sub_mobile:99886
```

Inheritance & Abstract

- Inheritance Basics
- **"super" keyword**
- Constructors in Inheritance
- Method Overriding
- Dynamic Method Dispatch
- Abstract Classes
- Object class

Inheritance & Abstract – "super" keyword

- The super keyword in Java is a reference variable which is used to refer immediate parent class object.
- Usage of Java super Keyword
  1) super can be used to **refer immediate parent class instance variable**.
  2) super can be used to **invoke immediate parent class method**.
  3) super() can be used to **invoke immediate parent class constructor**.

Syntax for constructor:

```
super(<para>);
```

Syntax for variable:

```
super.<var_name>
```

Syntax for function:

```
super.<fun_name>(<para>);
```

Inheritance & Abstract – "super" keyword

```
class data
{
        int a;
        data(int x)
        {
                a=x;
        }
        void disp()
        {
                System.out.println("from super a:"+a);
        }
}
```

```
from super a:5
from sub a:5
from sub b:6
```

```
class call1 extends data
{
        int b;
        call1(int x,int y)
        {
                super(x);
                b=y;
        }
        void display()
        {
                super.disp();
                System.out.println("from sub a:"+super.a);
                System.out.println("from sub b:"+b);
        }
}

class super1
{
public static void main(String args[])
{
        call1 obj = new call1(5,6);
        obj.display();
}
}
```

11

Inheritance & Abstract

- Inheritance Basics
- "super" keyword
- **Constructors in Inheritance**
- Method Overriding
- Dynamic Method Dispatch
- Abstract Classes
- Object class

12

Inheritance & Abstract – constructors in Inheritance

```
class A
{
        A() {
        System.out.println("Inside A's constructor.");
        }
}

class B extends A
{
        B() {
        System.out.println("Inside B's constructor.");
        }
}
```

```
class C extends B
{
        C() {
        System.out.println("Inside C's constructor.");
        }
}

class CallingCons
{
public static void main(String args[])
{
        C c = new C();
}
}
```

OUTPUT
Inside A's constructor
Inside B's constructor
Inside C's constructor

13

Inheritance & Abstract

- Inheritance Basics
- "super" keyword
- Constructors in Inheritance
- **Method Overriding**
- Dynamic Method Dispatch
- Abstract Classes
- Object class

14

Inheritance & Abstract – Method Overriding

- **Method Overriding (Run – Time Polymorphism) –**
  - Methods with same name in same scope with different behaviour.
  - ALL MUST BE SAME: return_type / No. of parameters / Parameter_types / sequence_of_Parameters

```
class A
{
        int i, j;
        A(int a, int b) {
        i = a;
        j = b;
        }
        void show()
        {
        System.out.println("i and j: " + i + " " + j);
        }
}
```

```
class B extends A
{
        int k;
        B(int a, int b, int c) {
        super(a, b);
        k = c;
        }
        void show() {
        System.out.println("k: " + k);
        }
}
class Override {
public static void main(String args[])
{
        B subOb = new B(1, 2, 3);
        subOb.show();
}
}
```

OUTPUT

??

Inheritance & Abstract – Method Overriding

- **As Sub Class object is created – it calls sub class method**
- **????? How to call Super class show() method ?????**

```
class A
{
        int i, j;
        A(int a, int b) {
        i = a;
        j = b;
        }
        void show()
        {
        System.out.println("i and j: " + i + " " + j);
        }
}
```

```
class B extends A
{
        int k;
        B(int a, int b, int c) {
        super(a, b);
        k = c;
        }
        void show() {
        System.out.println("k: " + k);
        }

class Override {
public static void main(String args[])

        B subOb = new B(1, 2, 3);
        subOb.show();
}
}
```

OUTPUT

k: 3

Inheritance & Abstract – Method Overriding

- As Sub Class object is created – it calls sub class method
- We can call superclass – override method using "super.<method>()".

```
class A
{
        int i, j;
        A(int a, int b) {
        i = a;
        j = b;
        }
        void show()
        {
        System.out.println("i and j: " + i + " " + j);
        }
}
```

```
class B extends A
{
        int k;
        B(int a, int b, int c) {
        super(a, b);
        k = c;
        }
        void show() {
        super.show();
        System.out.println("k: " + k);
        }

class Override {
public static void main(String args[])

        B subOb = new B(1, 2, 3);
        subOb.show();
}
}
```

OUTPUT
i and j : 1 2
k: 3

Inheritance & Abstract

- Inheritance Basics
- "super" keyword
- Constructors in Inheritance
- Method Overriding
- **Dynamic Method Dispatch**
- Abstract Classes
- Object class

Inheritance & Abstract – Dynamic Method Dispatch (DMD)

- Method overriding forms the basis for one of Java's most powerful concepts: **DYNAMIC METHOD DISPATCH.**
- **Dynamic method dispatch** is the **mechanism** by which *a call to an overridden method is resolved at run time*, rather than compile time.

OR

- **Overridden method calling problem resolved at run-time - the mechanism behind this is called Dynamic Method Dispatch.**
- Dynamic method dispatch is important because this is how Java implements run-time polymorphism.
- As its shown in last program.

Inheritance & Abstract – Dynamic Method Dispatch (DMD)

- Next Program - a superclass reference variable can refer to a subclass object.

```
class A
{
 /////
}
```

```
class B extends A
{
  /////
}
```

```
// Object creation and referencing
B obj_b = new B();
A obj_a;
obj_a = obj_b; // reference to sub class object
```

- When an **overridden method** is called through a superclass reference, Java determines which **version of that method to execute based upon the type of the object being referred to at the time the call occurs. (super class A's object – refer to – class B's Object).**
- This determination is made at **run time**.
- When different types of objects are referred to, different versions of an overridden method will be called.
- *In other words, it is the **type of the object being referred to** (not the type of the reference variable) that determines **which version of an overridden method will be executed**.*

Inheritance & Abstract – Dynamic Method Dispatch (DMD)

```
class A {
        void callme()
        {
        System.out.println("from A's method");
        }
}
```

```
class B extends A {
        void callme()
        {
        System.out.println("from B's method");
        }
}
```

```
class C extends A {
        void callme()
        {
        System.out.println("from C's method");
        }
}
```

```
class Dispatch {
public static void main(String args[])
{
        A a = new A();
        B b = new B();
        C c = new C();

        A r;

        r = a;
        r.callme();

        r = b;
        r.callme();

        r = c;
        r.callme();
}
}
```

Type of the object being referred to that determines which version of an overridden method will be executed.

OUTPUT

?

?

?

21

Inheritance & Abstract – Dynamic Method Dispatch (DMD)

```
class A {
        void callme()
        {
        System.out.println("from A's method");
        }
}
```

```
class B extends A {
        void callme()
        {
        System.out.println("from B's method");
        }
}
```

```
class C extends A {
        void callme()
        {
        System.out.println("from C's method");
        }
}
```

```
class Dispatch {
public static void main(String args[])
{
        A a = new A();
        B b = new B();
        C c = new C();

        A r;

        r = a;
        r.callme();

        r = b;
        r.callme();

        r = c;
        r.callme();
}
}
```

Type of the object being referred to that determines which version of an overridden method will be executed.

OUTPUT

from A's method

from B's method

from C's method

22

Inheritance & Abstract – Dynamic Method Dispatch (DMD)

- **Overridden methods**
  - Run Time Polymorphism.
  - Overridden methods are another way that Java implements the "one interface, multiple methods" aspect of polymorphism.
  - Here, the **superclass provides all elements** that a subclass can use directly.
  - It also defines **those methods that the derived class must implement** on its own.
  - Thus, by combining inheritance with overridden methods, **a superclass can define the methods that will be used by all of its subclasses.**
  - DMD Mechanism provides reuse and robustness in JAVA Code.

Inheritance & Abstract – Dynamic Method Dispatch (DMD)

- **Program**

- **Figure – super class**
- **Rectangle – sub class**
- **Triangle – sub class**

Figure - super class
double area() – function to be override

Rectangle - sub class
double area() – function override to find its area

Triangle - sub class
double area() – function override to find its area

Inheritance & Abstract – Dynamic Method Dispatch (DMD)

- **Program**

- **Figure – super class**
- **Rectangle – sub class**
- **Triangle – sub class**

Figure - super class double area() – function to be overloaded

Rectangle - sub class double area() – function overloaded to find its area

Triangle - sub class double area() – function overloaded to find its area

```
class Figure {
double dim1,dim2;
 Figure(double a, double b)
 {
 dim1 = a;
 dim2 = b;
 }
 double area() {
System.out.println("from Figure class.");
 return 0;
 }
}
```

```
class Rectangle extends Figure
{
 Rectangle(double a, double b)
 {
            super(a, b);
 }

 double area() {
  System.out.println(“from Rectangle.");
  return dim1 * dim2;
 }
}
```

```
class Triangle extends Figure
{
 Triangle(double a, double b)
 {
            super(a, b);
 }

 double area() {
  System.out.println(“From Triangle.");
  return dim1 * dim2 / 2;
 }
}
```


Inheritance & Abstract – Dynamic Method Dispatch (DMD)

```
class Figure {
double dim1,dim2;
 Figure(double a, double b)
 {
 dim1 = a;
 dim2 = b;
 }
 double area() {
System.out.println("from Figure class.");
 return 0;
 }
}
```

```
class Rectangle extends Figure
{
 Rectangle(double a, double b)
 {
            super(a, b);
 }
 double area() {
  System.out.println(“from Rectangle.");
  return dim1 * dim2;
 }
}
```

```
class Triangle extends Figure
{
 Triangle(double a, double b)
 {
            super(a, b);
 }
 double area() {
  System.out.println(“From Triangle.");
  return dim1 * dim2 / 2;
 }
}
```

```
class FindAreas {
public static void main(String args[]) {
Figure f = new Figure(10, 10);
Rectangle r = new Rectangle(9, 5);
Triangle t = new Triangle(10, 8);

Figure f1;
f1 = r;
System.out.println(f1.area());
f1 = t;
System.out.println(f1.area());
f1 = f;
System.out.println(f1.area());
}
}
```

Inheritance & Abstract – Dynamic Method Dispatch (DMD)

```
class Figure {
double dim1,dim2;
 Figure(double a, double b)
 {
 dim1 = a;
 dim2 = b;
 }
 double area() {
System.out.println("from Figure class.");
 return 0;
 }
}
```

OUTPUT
from Rectangle.
Area is 45
from Triangle.
Area is 40
from Figure class.
Area is 0

```
class Rectangle extends Figure
{
 Rectangle(double a, double b)
 {
            super(a, b);
 }
 double area() {
  System.out.println("from Rectangle.");
  return dim1 * dim2;
 }
}
```

```
class Triangle extends Figure
{
 Triangle(double a, double b)
 {
            super(a, b);
 }
 double area() {
  System.out.println("From Triangle.");
  return dim1 * dim2 / 2;
 }
}
```

```
class FindAreas {
public static void main(String args[]) {
Figure f = new Figure(10, 10);
Rectangle r = new Rectangle(9, 5);
Triangle t = new Triangle(10, 8);

Figure f1;
f1 = r;
System.out.println(f1.area());
f1 = t;
System.out.println(f1.area());
f1 = f;
System.out.println(f1.area());
}
}
```


Inheritance & Abstract – Dynamic Method Dispatch (DMD)

- **Program**

- **Figure – super class**
- **Rectangle – sub class**
- **Triangle – sub class**

Figure - super class double area() – function to be overloaded

Rectangle - sub class double area() – function overloaded to find its area

Triangle - sub class double area() – function overloaded to find its area

- **From Output – Superclass function is only to definition for subclass override function.**
- **So, only definition is used in subclass.**
- **Function Body is never being used.**
- **Super class function body is never being used, as it's used for only definition of function**
- **This Leads to new concept called ABSTRACT CLASSES.**

OUTPUT
from Rectangle.
Area is 45
from Triangle.
Area is 40
from Figure class.
Area is 0

Inheritance & Abstract

- Inheritance Basics
- "super" keyword
- Constructors in Inheritance
- Method Overriding
- Dynamic Method Dispatch
- **Abstract Classes**
- Object class

29

Inheritance & Abstract – Abstract classes

- Super-class functions will not be used for any execution, it's only used as a definition purpose.
- It is possible to have a Super – Class with only structure of methods without any body.
- **It is also called structure of abstraction without providing complete implementation.**

- **Such Classes are called Abstract Classes in JAVA.**
- ***Important:***
  - **Class must be prefixed by "abstract" keyword.**
  - **At least one method must be "abstract method" in class.**
  - **Abstract class may have non-abstract methods.**
  - **All Abstract – Methods must be Overridden in each sub-class.**
  - **Object of abstract class can be declared – not to create with "new" keyword.**

```
abstract class data
{

abstract void f1(int,int);

void show()
{
System.out.println("from abstarct class");
}

}
```

30

Inheritance & Abstract – Abstract classes

- **Program**

- **Figure – super class**
- **Rectangle – sub class**
- **Triangle – sub class**

Figure - super class
double area() – function to be override

Rectangle - sub class
double area() – function override to find its area

Triangle - sub class
double area() – function override to find its area

```
abstract class Figure {
double dim1,dim2;
 Figure(double a, double b)
 {
 dim1 = a;
 dim2 = b;
 }
 abstract double area() ;
}
```

```
class Rectangle extends Figure
{
 Rectangle(double a, double b)
 {
            super(a, b);
 }

 double area() {
  System.out.println("from Rectangle.");
  return dim1 * dim2;
 }
}
```

```
class Triangle extends Figure
{
 Triangle(double a, double b)
 {
            super(a, b);
 }

 double area() {
  System.out.println("From Triangle.");
  return dim1 * dim2 / 2;
 }
}
```


Inheritance & Abstract – Dynamic Method Dispatch (DMD)

```
abstract class Figure {
double dim1,dim2;
 Figure(double a, double b)
 {
 dim1 = a;
 dim2 = b;
 }
 abstract double area() ;
}
```

```
class Rectangle extends Figure
{
 Rectangle(double a, double b)
 {
            super(a, b);
 }
 double area() {
  System.out.println("from Rectangle.");
  return dim1 * dim2;
 }
}
```

```
class Triangle extends Figure
{
 Triangle(double a, double b)
 {
            super(a, b);
 }
 double area() {
  System.out.println("From Triangle.");
  return dim1 * dim2 / 2;
 }
}
```

```
class FindAreas {
public static void main(String args[]) {
Rectangle r = new Rectangle(9, 5);
Triangle t = new Triangle(10, 8);

Figure f1;
f1 = r;
System.out.println(f1.area());
f1 = t;
System.out.println(f1.area());
}
}
```

OUTPUT
from Rectangle.
Area is 45
from Triangle.
Area is 40

Inheritance & Abstract

- Inheritance Basics
- "super" keyword
- Constructors in Inheritance
- Method Overriding
- Dynamic Method Dispatch
- Abstract Classes
- **Object class**

Inheritance & Abstract – Object Class

Method	Purpose
Object clone( )	Creates a new object that is the same as the object being cloned.
boolean equals(Object *object*)	Determines whether one object is equal to another.
void finalize( )	Called before an unused object is recycled.
Class getClass( )	Obtains the class of an object at run time.
int hashCode( )	Returns the hash code associated with the invoking object.
void notify( )	Resumes execution of a thread waiting on the invoking object.
void notifyAll( )	Resumes execution of all threads waiting on the invoking object.
String toString( )	Returns a string that describes the object.
void wait( ) void wait(long *milliseconds*) void wait(long *milliseconds*, int *nanoseconds*)	Waits on another thread of execution.